Discrete Maths

240-213, Semester 2, 2022-2023

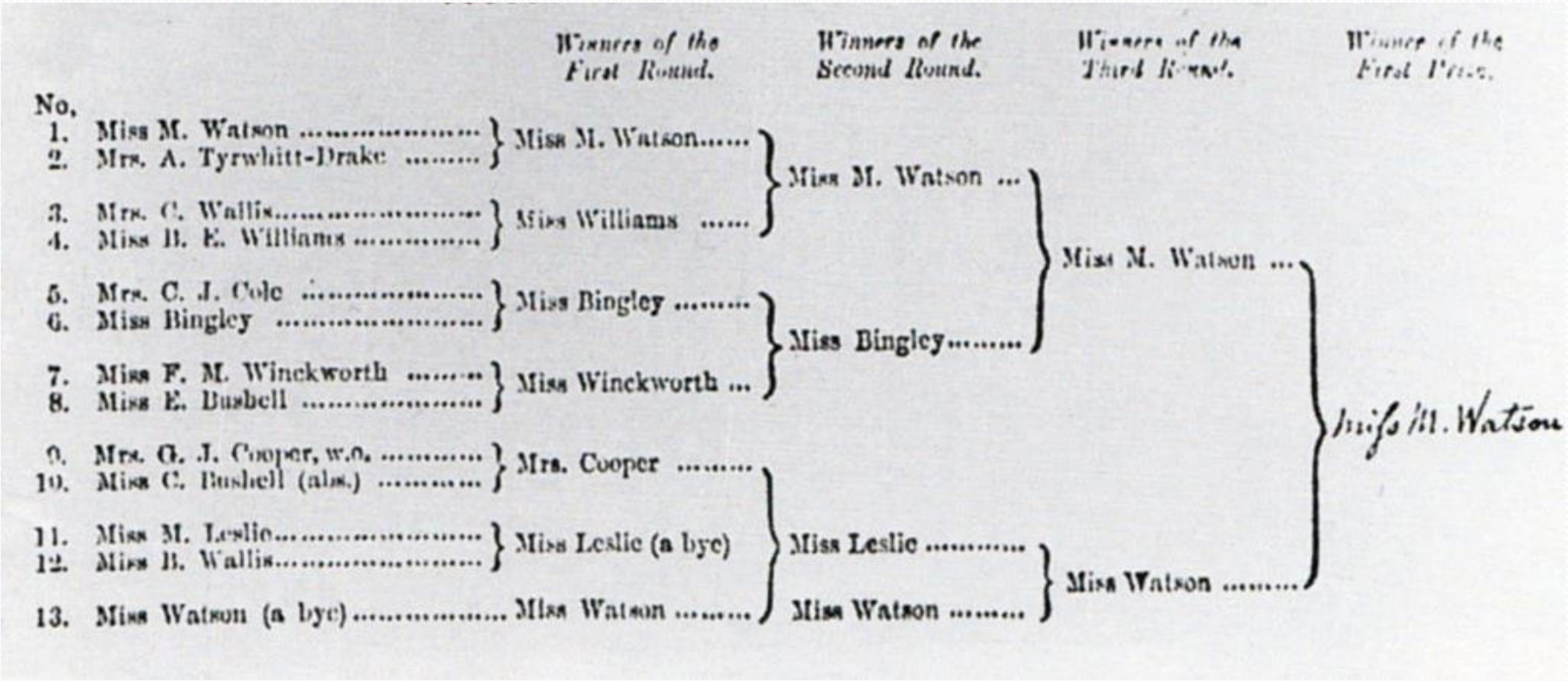
10. Trees

- Objective
  - introduce more unusual tree algorithms and techniques
    - you already know about binary (search) trees

Overview

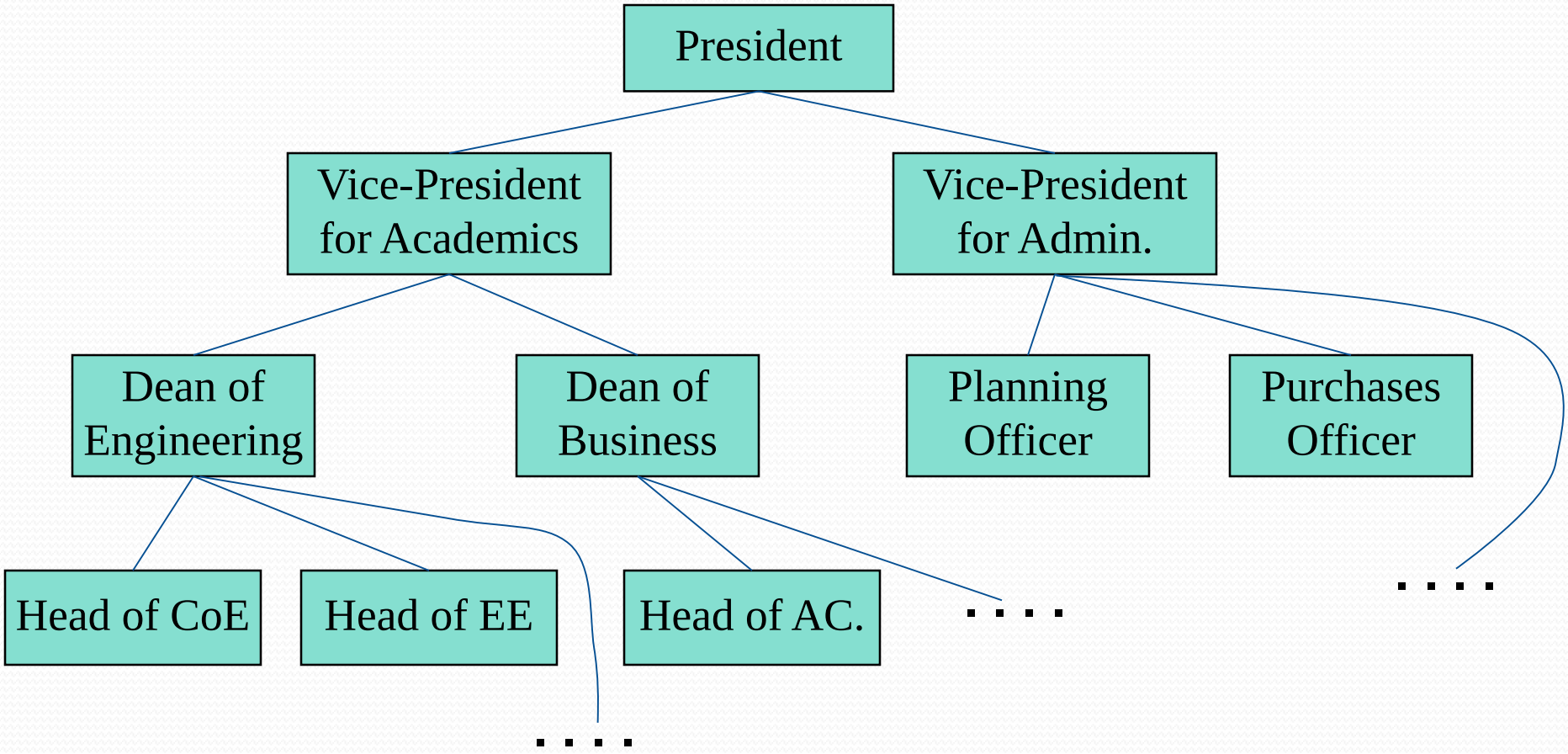
1. Uses of Trees
2. (Rooted) Tree Terminology
3. Spanning Trees
4. Minimal Spanning Trees
5. The Travelling Salesman Problem Again
6. More Information

1. Uses of Trees

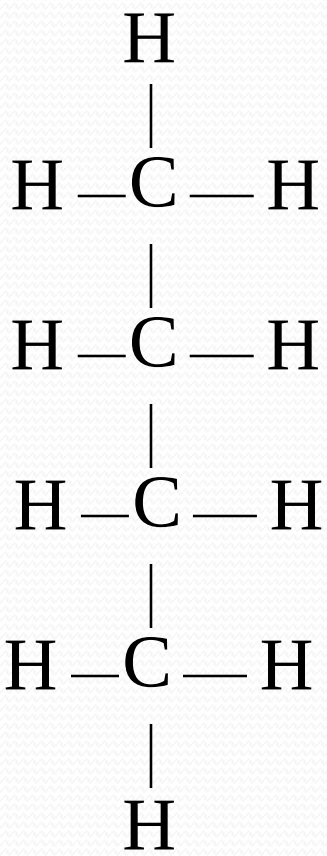


Wimbledon Ladies Championship, 1884

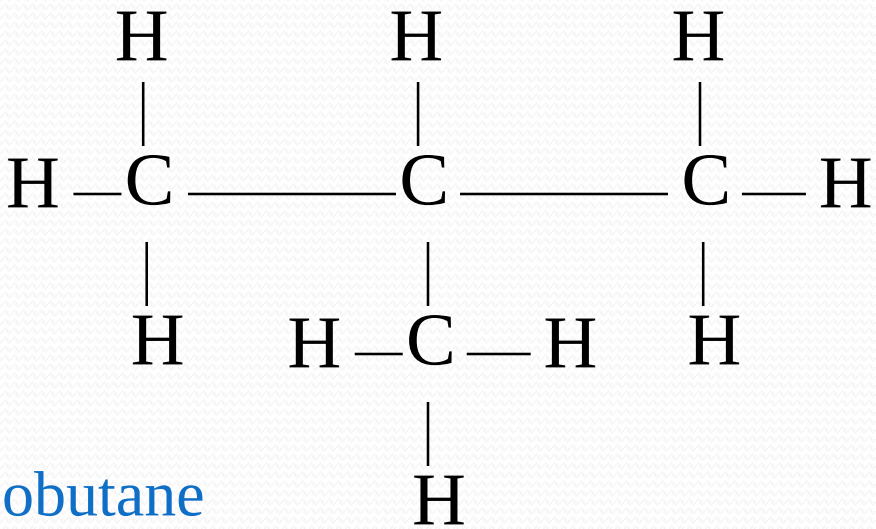
Organizational Chart



Saturated Hydrocarbons

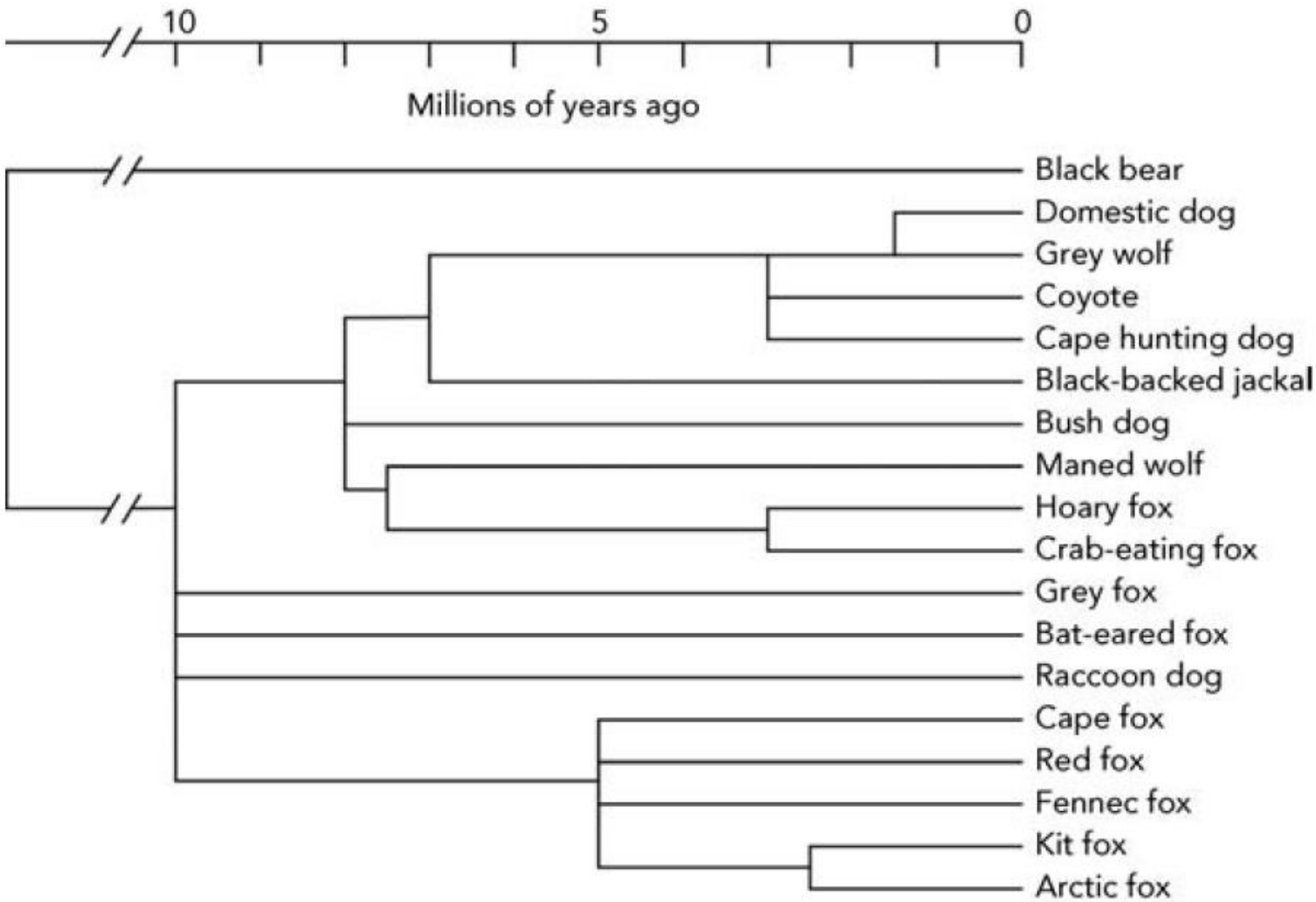


Butane



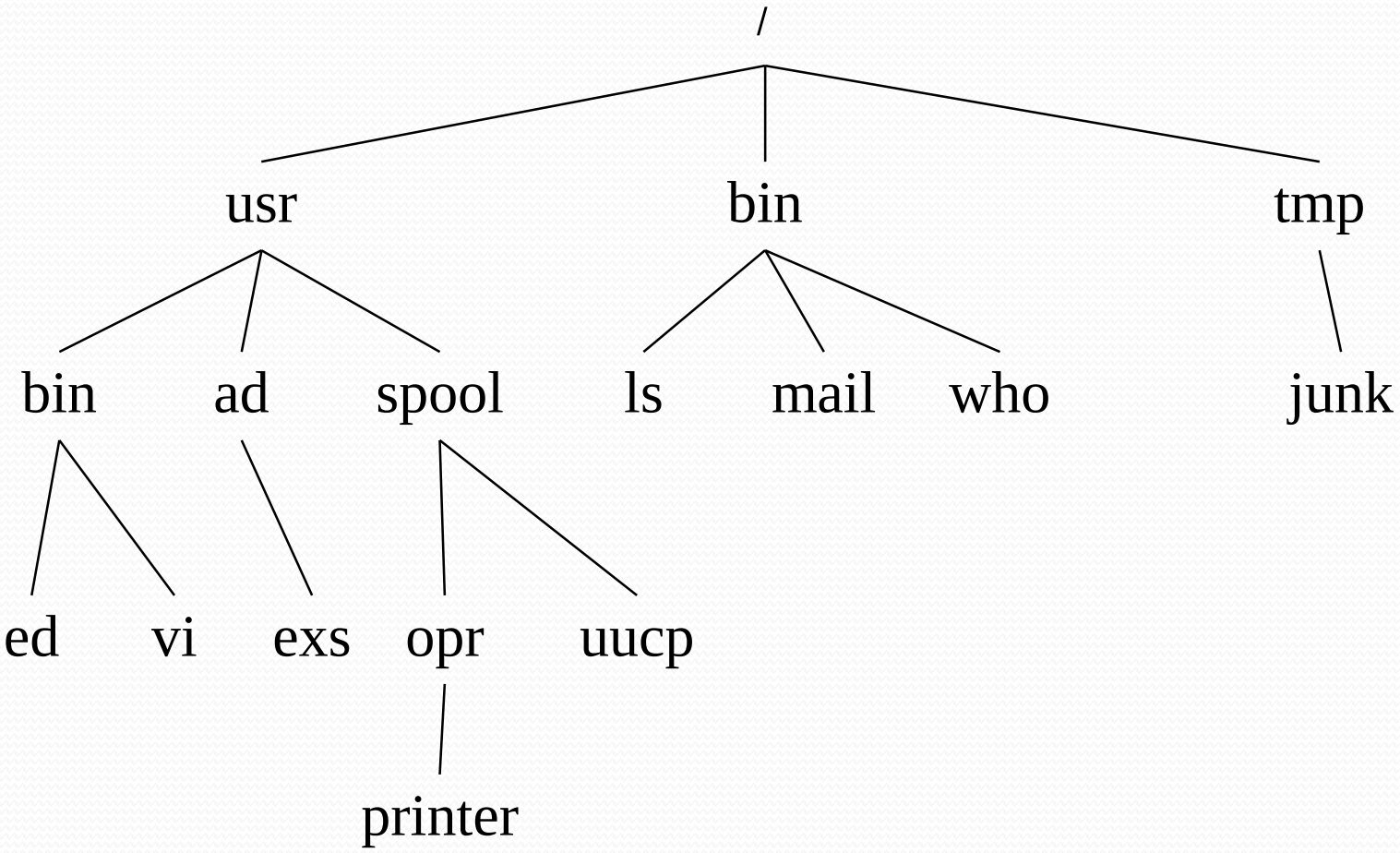
Isobutane

- Non-rooted (*free*) trees
  - a free tree is a graph with no cycles



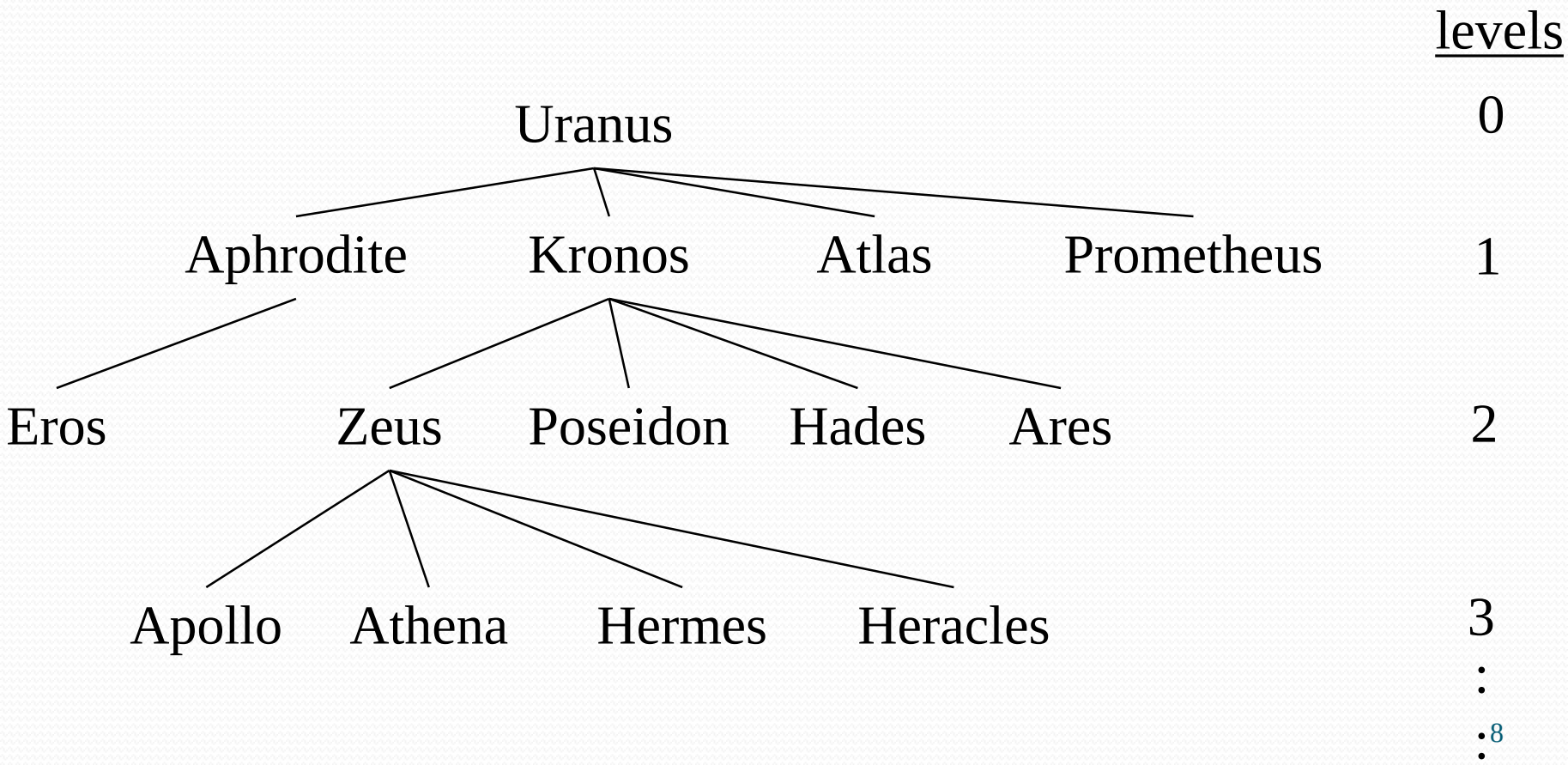
Cladogram for the domestic dog.

A Computer File System



2. (Rooted) Tree Terminology

- e.g. Part of the ancient Greek god family:



Some Definitions

- Let T be a tree with root v_0.
- Suppose that x, y, z are verticies in T.
- $(v_0, v_1, \ldots, v_n)$ is a *simple path* in T (no loops).

- a) v_{n-1} is the *parent* of v_n.
- b) $v_0, \ldots, v_{n-1}$ are *ancestors* of v_n
- c) v_n is a *child* of v_{n-1}

continued

- d) If x is an ancestor of y, then y is a *descendant* of x.
- e) If x and y are children of z, then x and y are *siblings*.
- f) If x has no children, then x is a terminal vertex (or a *leaf*).
- g) If x is not a terminal vertex, then x is an internal (or *branch*) vertex.

continued

- h) The *subtree* of T rooted at x is the graph with vertex set V and edge set E
  - V contains x and all the descendents of x
  - E = {e | e is an edge on a simple path from x to some vertex in V}

- i) The *length* of a path is the **number of edges** it uses, not verticies.

continued

- j) The *level* of a vertex x is the length of the simple path from the root to x.

- k) The *height* of a vertex x is the length of the simple path from x to the farthest leaf
  - the *height of a tree* is the height of its root

- l) A tree where every internal vertex has exactly m children is called a full *m-ary tree*.

Applied to the Example

- The *root* is Uranus.
- A *simple path* is {Uranus, Aphrodite, Eros}

- The *parent* of Eros is Aphrodite.
- The *ancestors* of Hermes are Zeus, Kronos, and Uranus.
- The *children* of Zeus are Apollo, Athena, Hermes, and Heracles.

continued

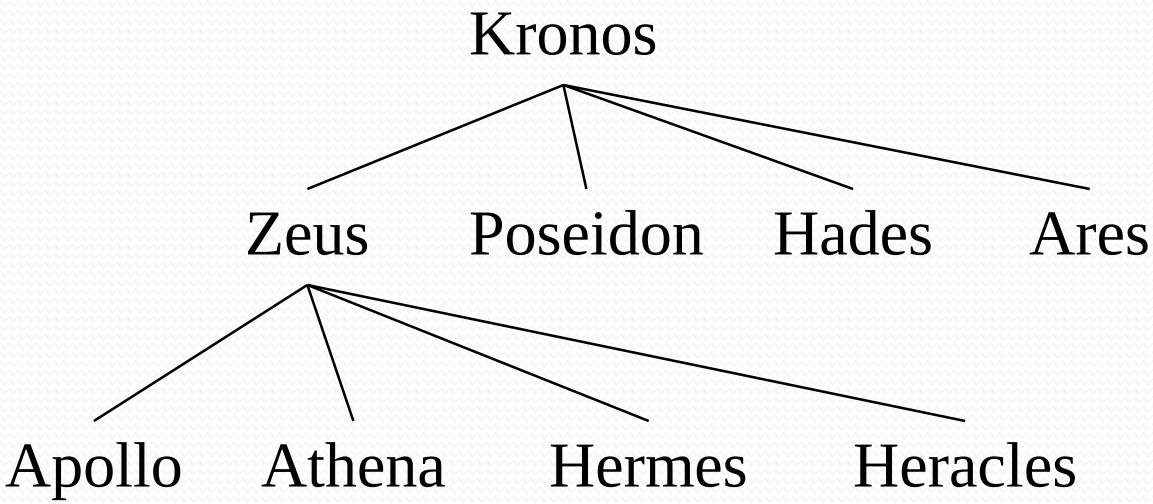
- The *descendants* of Kronos are Zeus, Poseidon, Hades, Ares, Apollo, Athena, Hermes, and Heracles.

- The *leaves* (terminal verticies) are Eros, Apollo, Athena, Hermes, Heracles, Poseidon, Hades, Ares, Atlas, and Prometheus.

- The *branches* (internal verticies) are Uranus, Aphrodite, Kronos, and Zeus.

continued

- The *subtree* rooted at Kronos:

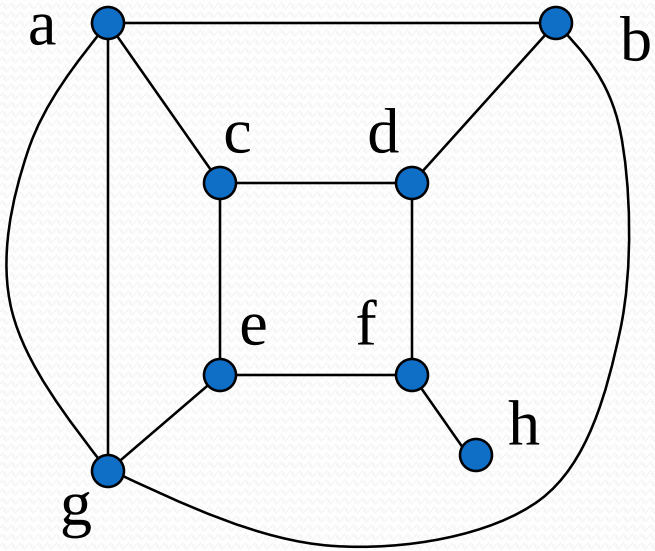


continued

- The *length* of the path {Uranus, Aphrodite, Eros} is 2 (not 3).
- The *level* of Ares is 2.
- The *height* of the tree is 3.

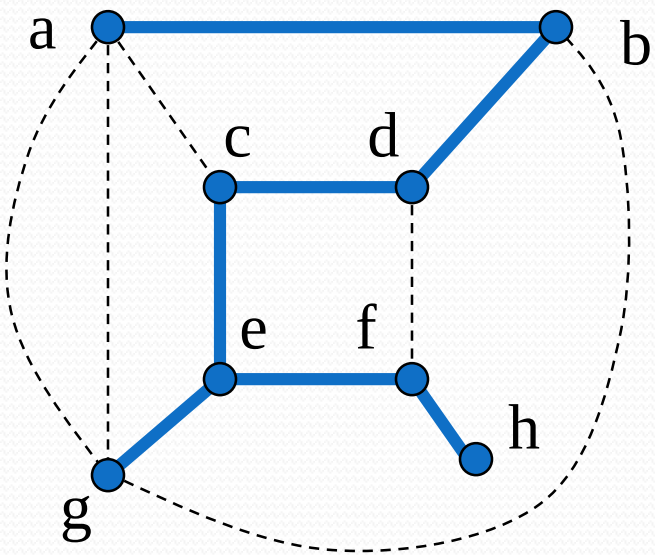
3. Spanning Trees

- A spanning tree T is a subgraph of a graph G which contains all the verticies of G.
- Example graph G:



continued

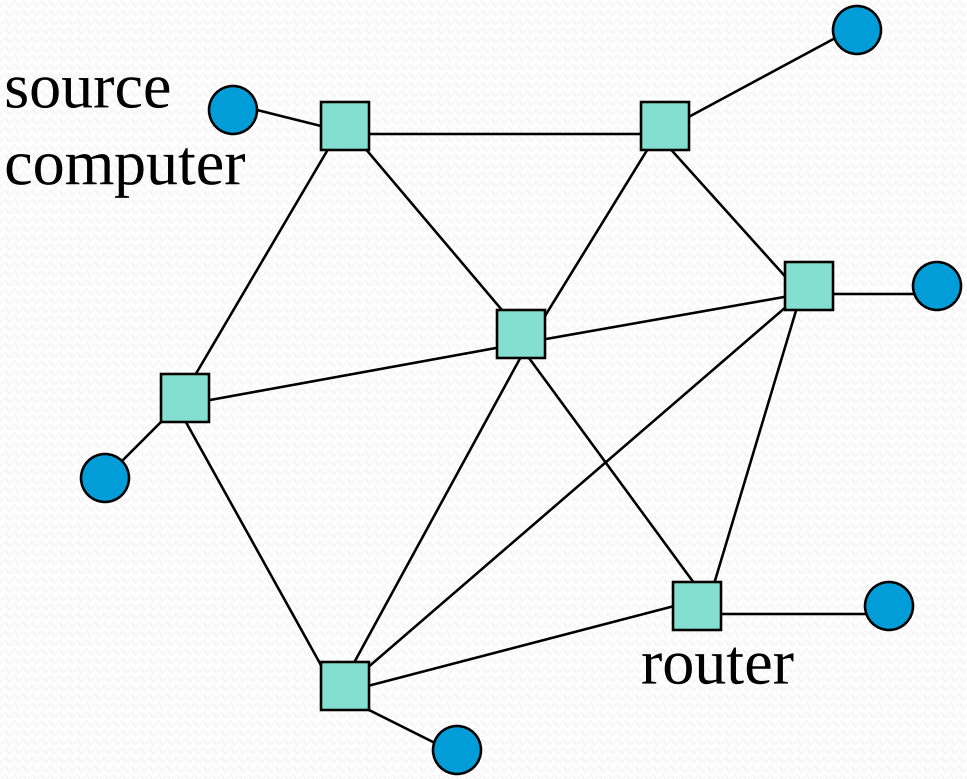
- One possible spanning tree:



the tree is drawn with thick lines

3.1. Example: IP Multicasting

- A network of computers and routers:



continued

- How can a packet (message) be sent from the source computer to every other computer?

- The *inefficient* way is to use *broadcasting*
  - send a copy along every link, and have each router do the same
  - each router and computer will receive many copies of the same packet
  - loops may mean the packet never disappears!

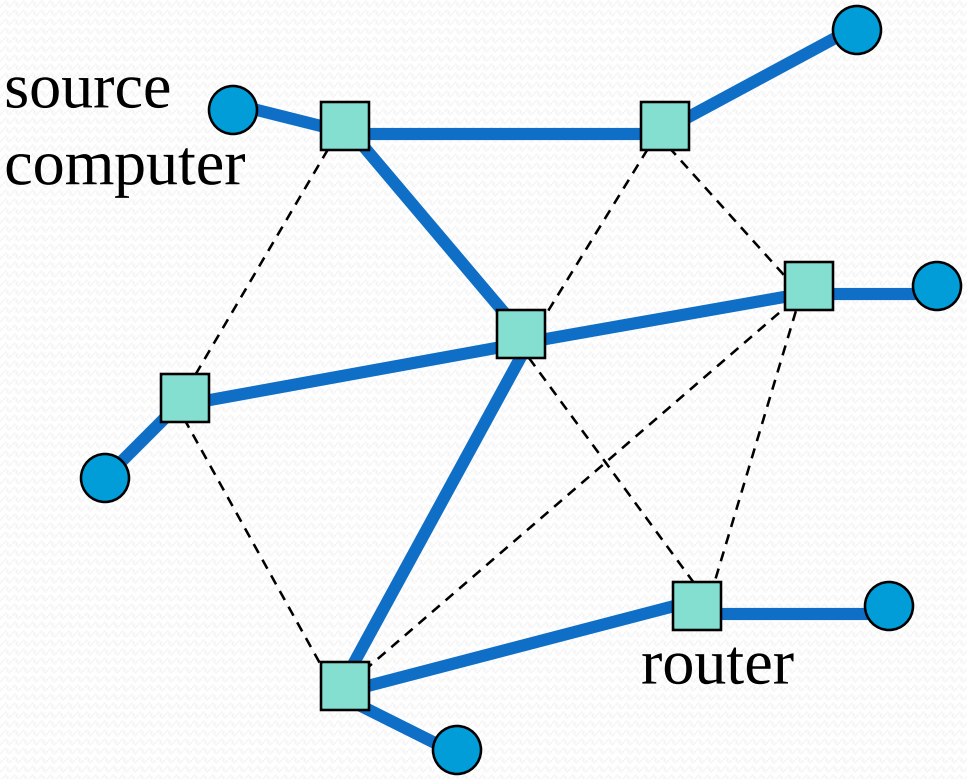
continued

- IP multicasting is an *efficient* solution
  - send a single packet to one router
  - have the router send it to 1 or more routers in such a way that a computer never receives the packet more than once

- This behaviour can be represented by a spanning tree.

continued

- The spanning tree for the network:



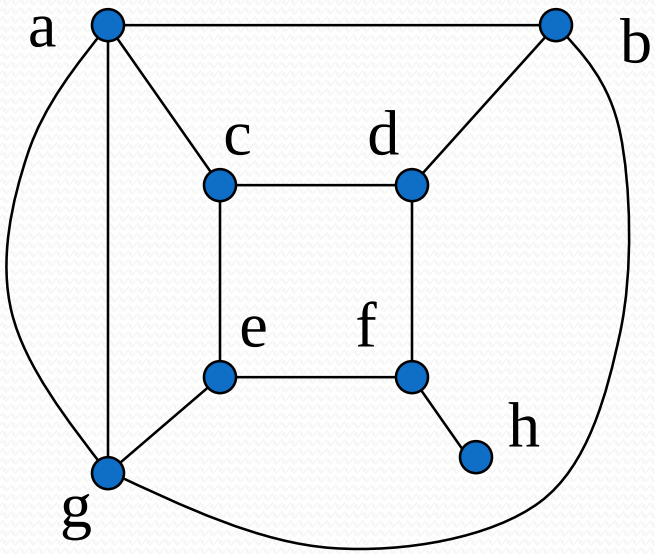
the tree is drawn with thick lines

3.2. Finding a Spanning Tree

- There are two main types of algorithms:
  - breadth-first search
  - depth-first search

3.3. Breadth-First Search (BFS)

- Process all the verticies at a given level before moving to the next level.
- Example graph G (again):

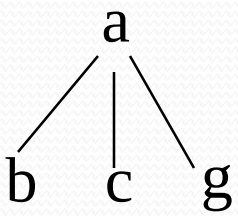


Informal Algorithm

- 1) Put the verticies into an ordering
  - e.g. {a, b, c, d, e, f, g, h}

- 2) Select a vertex, add it to the spanning tree T: e.g. a

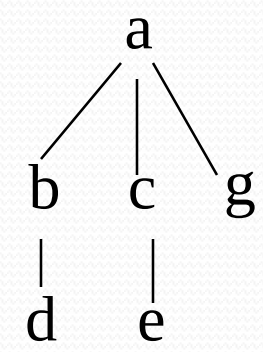
- 3) Add to T all edges (a,X) and X verticies that do not create a cycle in T
  - i.e. (a,b), (a,c), (a,g)
    T = {a, b, c, g}



continued

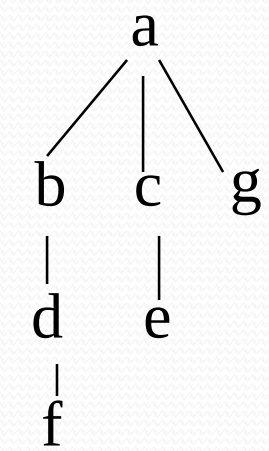
- Repeat step 3 on the verticies just added, these are on level 1
  - i.e. b: add (b,d)
    c: add (c,e)
    g: nothing
    T = {a,b,c,d,e}

level 1



- Repeat step 3 on the verticies just added, these are on level 2
  - i.e. d: add (d,f)
    e: nothing
    T = {a,b,c,d,e,f}

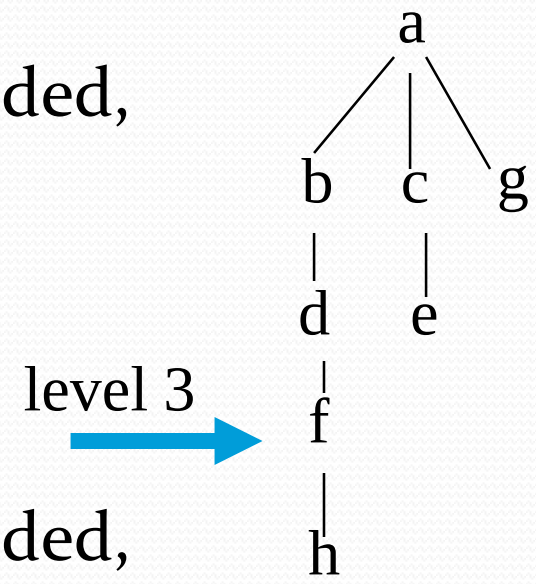
level 2



continued

- Repeat step 3 on the verticies just added, these are on level 3
  - i.e. f: add (f,h)

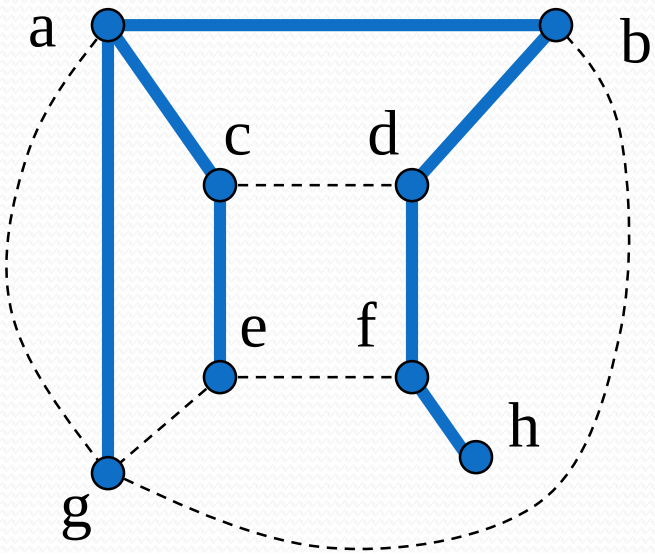
    T = {a,b,c,d,e,f,h}



- Repeat step 3 on the verticies just added, these are on level 4
  - i.e. h: nothing, so stop

continued

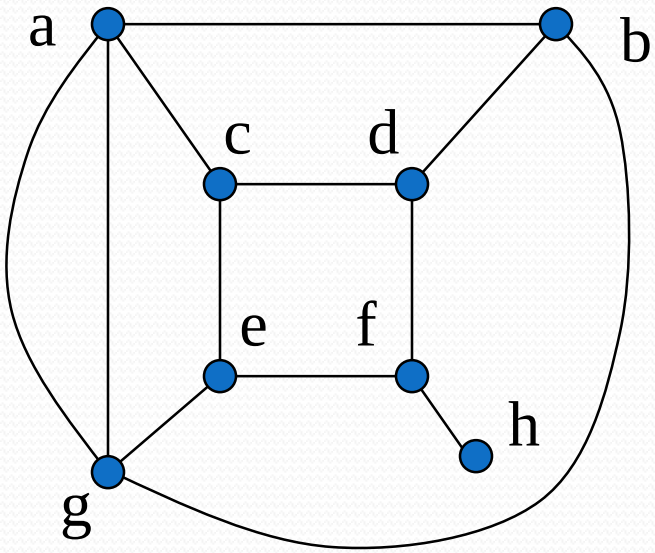
- Resulting spanning tree:



a different spanning tree from the earlier solution

3.4. Depth-First Search (DFS)

- Process all the verticies down one path, then *backtrack* (go back) to verticies along other paths.
- Example graph G (again):



Informal Algorithm

- 1) Put the verticies into an ordering
  - e.g. {a, b, c, d, e, f, g, h}

- 2) Select a vertex, add it to the spanning tree T: e.g. a

- 3) Add the edge (a,X) where X is the smallest vertex in the ordering, and does not make a cycle in T
  - i.e. (a,b), T = {a, b}

a
|
b

continued

- 4) Repeat step 3 with the new vertex, until there is no possible new vertex
  - i.e. add the edges (b,d) (d,c) (c,e) (e,f) (f,h)
    T = {a,b,d,c,e,f,h}

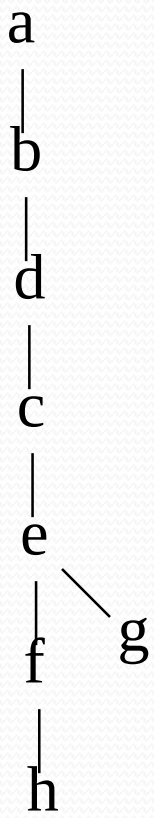
- 5) At this point, there is no (h,X), so backtrack to a vertex that does have another edge:
  - parent of h == f
    but there is no new (f,X) to add, so backtrack

a
|
b
|
d
|
c
|
e
|
f
|
h

continued

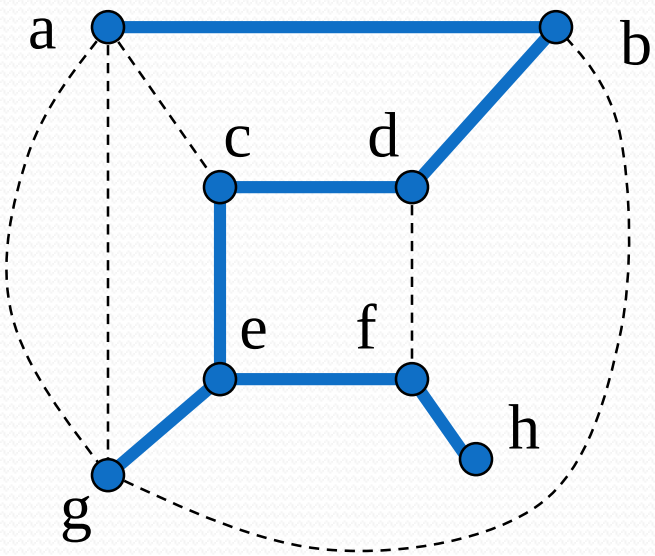
- parent of f == e
- there is an (e,g) to add, so repeat step 3 with e

- 6) After g is added, there are no further verticies to add, so stop.



continued

- Resulting spanning tree:



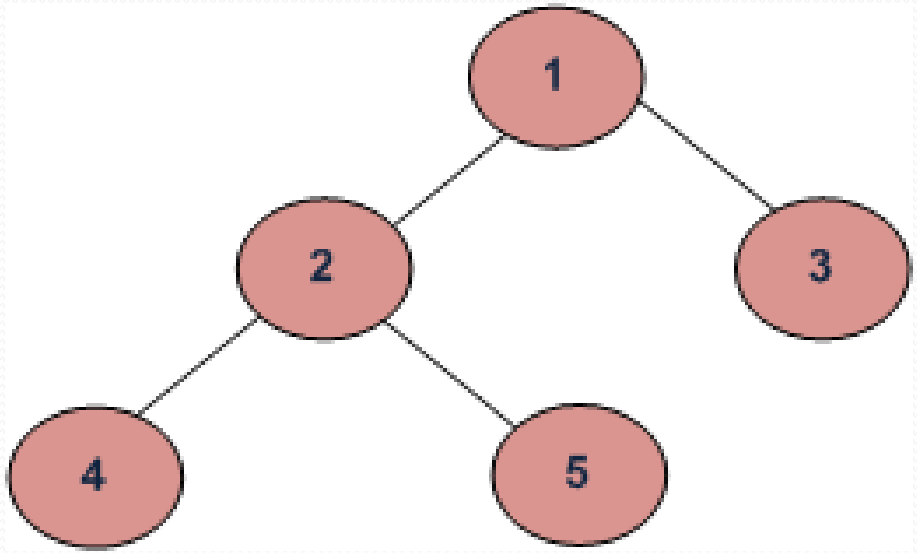
a different spanning tree from the breadth-first solution

3.5. Different Types of DFS

- (a) Inorder (Left, Root, Right): 4, 2, 5, 1, 3
- (b) **Preorder** (Root, Left, Right): 1, 2, 4, 5, 3
- (c) Postorder (Left, Right, Root): 4, 5, 2, 3, 1

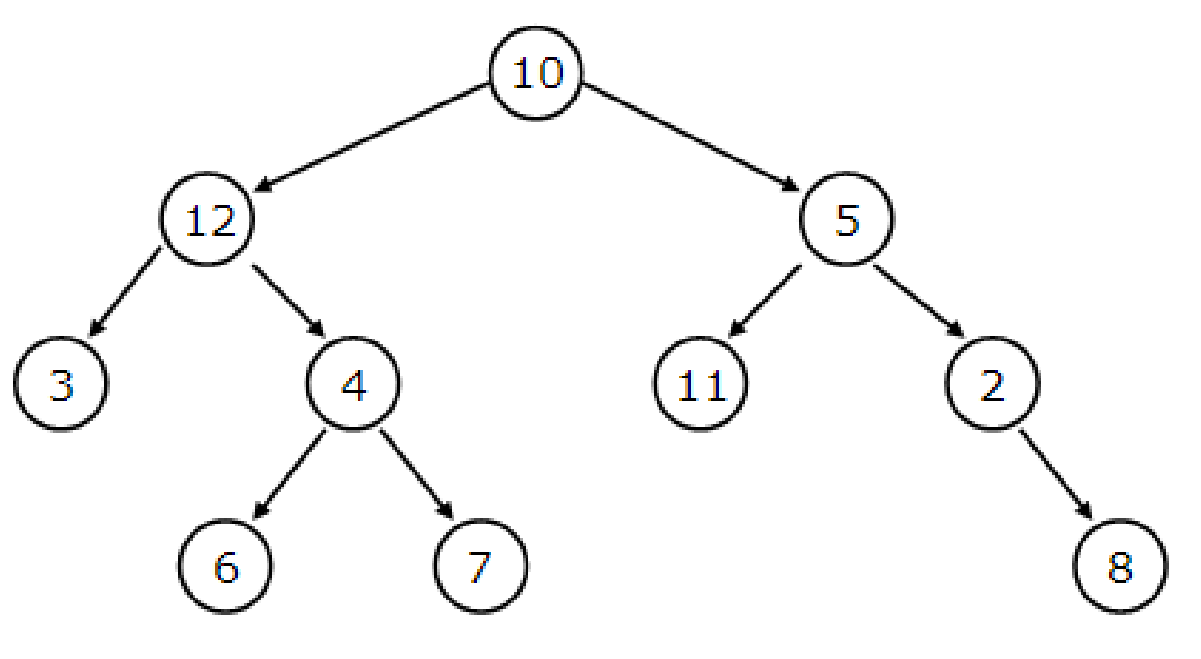
we just used this one on a graph

To keep things simple, we'll only look at **DFS for a binary tree**.



Another Example

- Inorder (Left, Root, Right): 3, 12, 6, 4, 7, 10, 11, 5, 2, 8
- **Preorder** (Root, Left, Right): 10, 12, 3, 4, 6, 7, 5, 11, 2, 8
- Postorder (Left, Right, Root): 3, 6, 7, 4, 12, 11, 8, 2, 5, 10



C Code

searchTrees.c

creates a binary search tree

```c
typedef struct Node {
  int data;
  struct Node *left;
  struct Node *right;
}  Node;


Node *makeNode(int d)
{
  printf("Adding %d\n", d);
  Node *n = (Node *) malloc(sizeof(Node));
  n->data = d;
  n->left = NULL;
  n->right = NULL;
  return n;
}
```

```c
void insert(Node *node, int d)
{
  if (d == node->data)
    printf("Node %d already exists\n");
  else if (d < node->data) {
    // Recurse to left subtree
    if (node->left == NULL)
      node->left = makeNode(d);
    else
      insert(node->left, d);
  }
  else {
    // Recurse to right subtree
    if (node->right == NULL)
      node->right = makeNode(d);
    else
      insert(node->right, d);
  }
}  // end of insert()
```

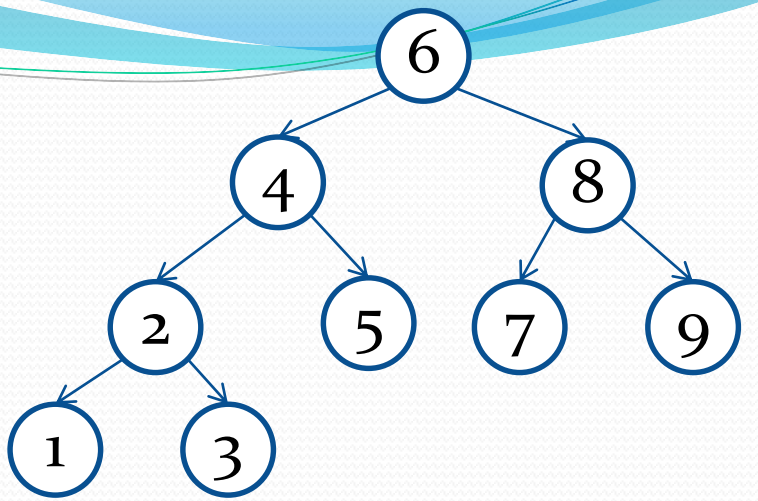
```c
int main()
{
  printf("Constructing a binary search tree:\n");
  Node *root = makeNode(6);
  insert(root, 4);
  insert(root, 2);
  insert(root, 1);
  insert(root, 3);
  insert(root, 5);
  insert(root, 8);
  insert(root, 7);
  insert(root, 9);

  printf("\nIn-order traversal:\n  ");
  inOrder(root);

  printf("\n\nPre-order traversal:\n  ");
  preOrder(root);

  printf("\n\nPost-order traversal\n  ");
  postOrder(root);

  return 0;
}
```




```
Constructing a binary search tree:
Adding 6
Adding 4
Adding 2
Adding 1
Adding 3
Adding 5
Adding 8
Adding 7
Adding 9

In-order traversal:
  1  2  3  4  5  6  7  8  9

Pre-order traversal:
  6  4  2  1  3  5  8  7  9

Post-order traversal
  1  3  2  5  4  7  9  8  6
>
```

Inorder (Left, Root, Right):
Preorder (Root, Left, Right)
Postorder (Left, Right, Root)

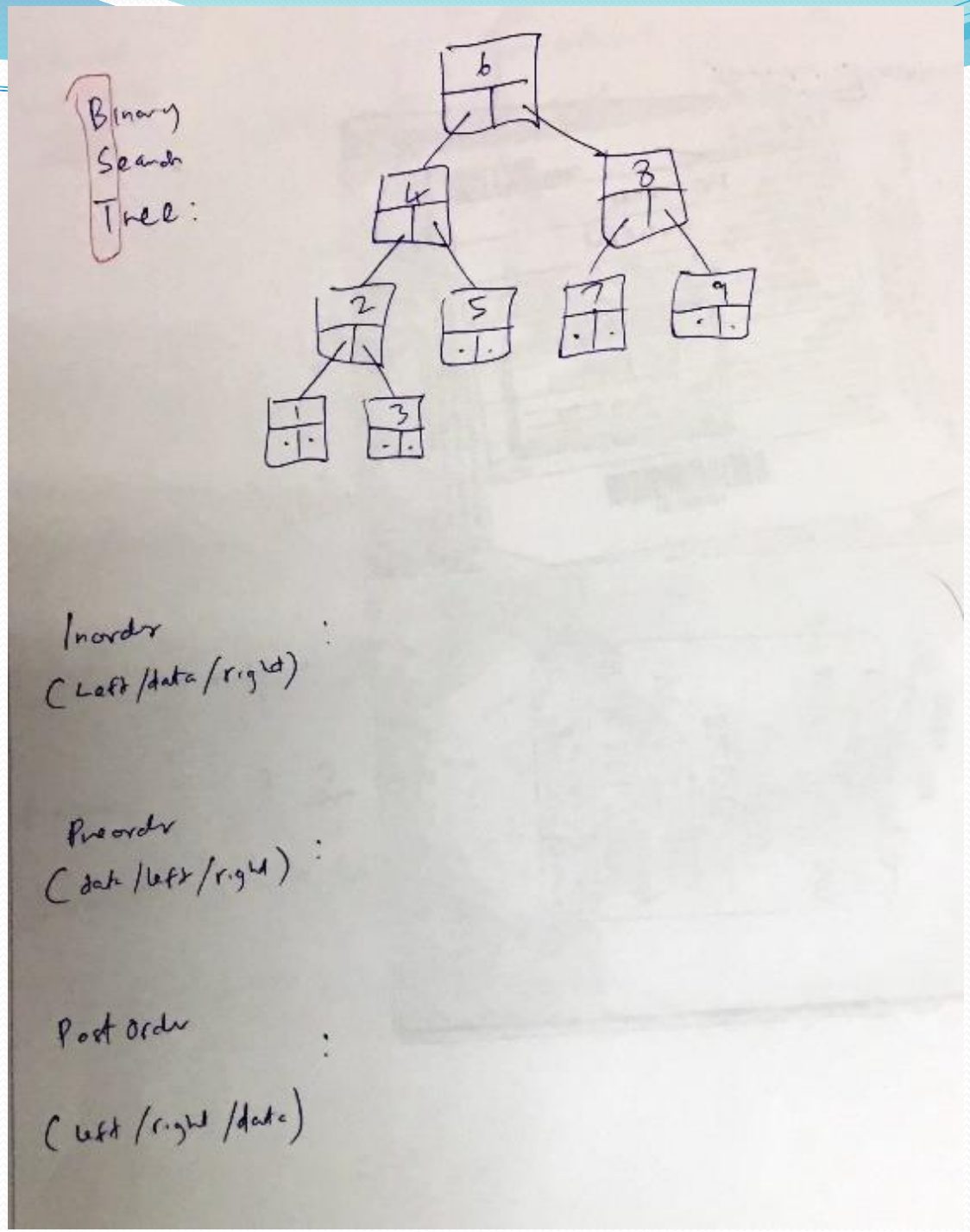
Three DFS Searches

```
void inOrder(Node *root)
// Inorder (Left, Root, Right)
{
  if (root == NULL)
    return;
  else {
    inOrder(root->left); // left
    printf("%d  ", root->data);
    inOrder(root->right); // right
  }
}
```

```
void preOrder(Node *root)
// Preorder (Root, Left, Right)
{
  if (root == NULL)
    return;
  else {
    printf("%d  ", root->data);
    preOrder(root->left); // left
    preOrder(root->right); // right
  }
}
```

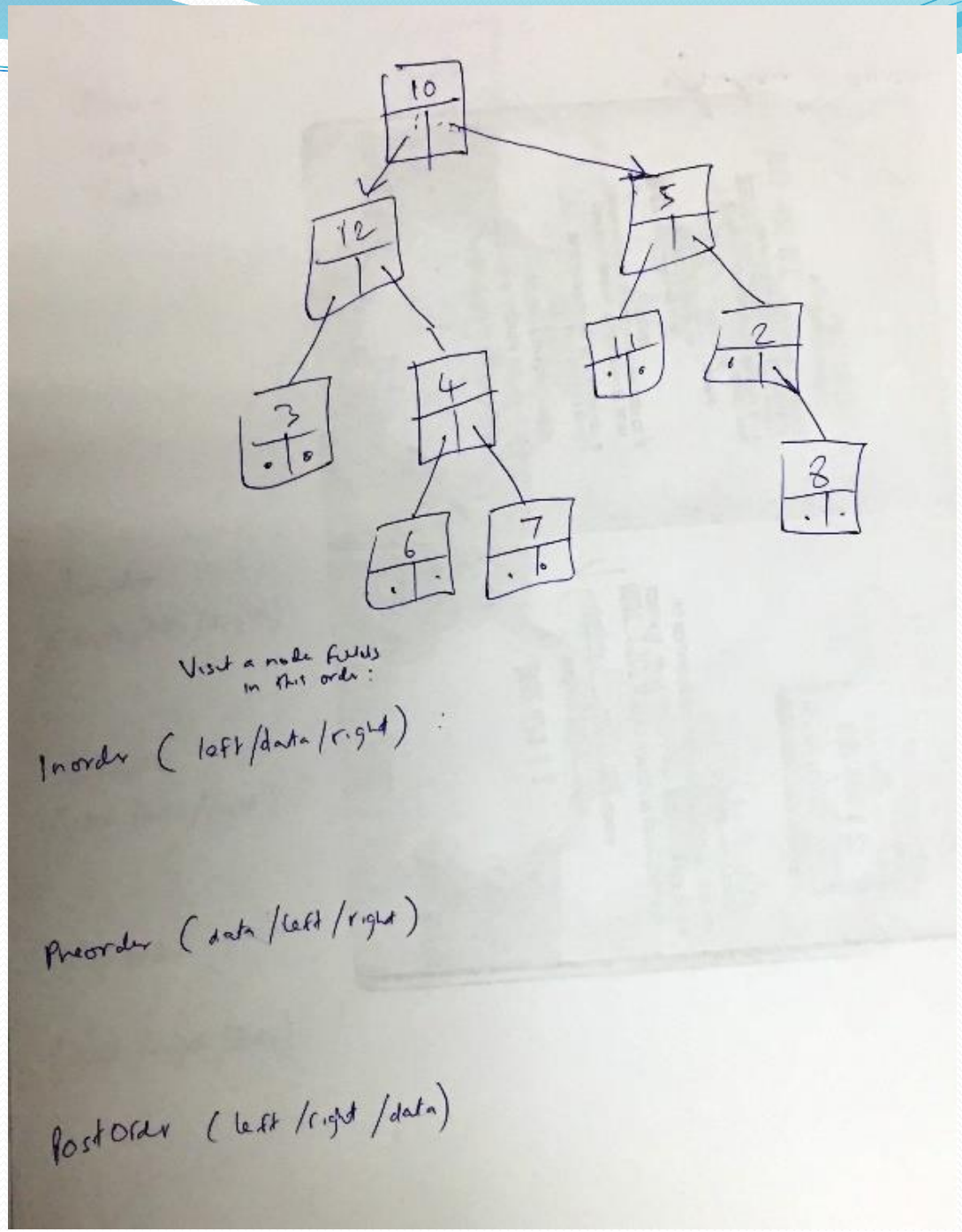
```
void postOrder(Node *root)
// Postorder (Left, Right, Root)
{
  if (root == NULL)
    return;
  else {
    postOrder(root->left); // left
    postOrder(root->right); // right
    printf("%d  ", root->data);
  }
}
```

Three Types of DFS Using C



Another C Example

not a BST tree,
but it is a binary tree



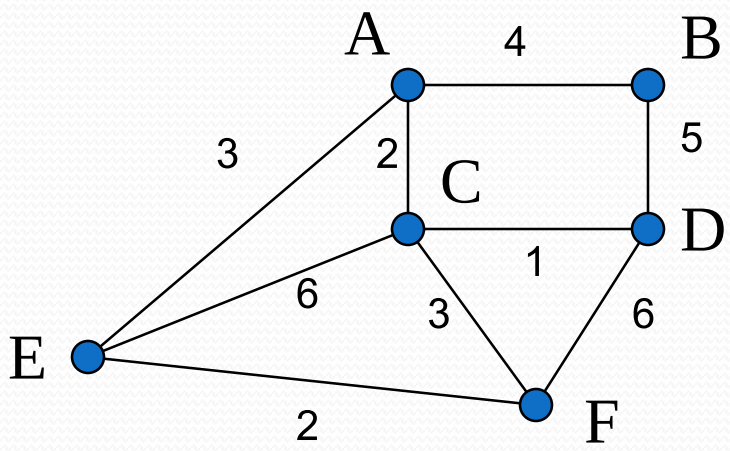
DFS Code for a Graph?

- One difference in DFS code for searching a graph instead of a tree is to **avoid going into loops**
  - you need to keep a record of already visited nodes

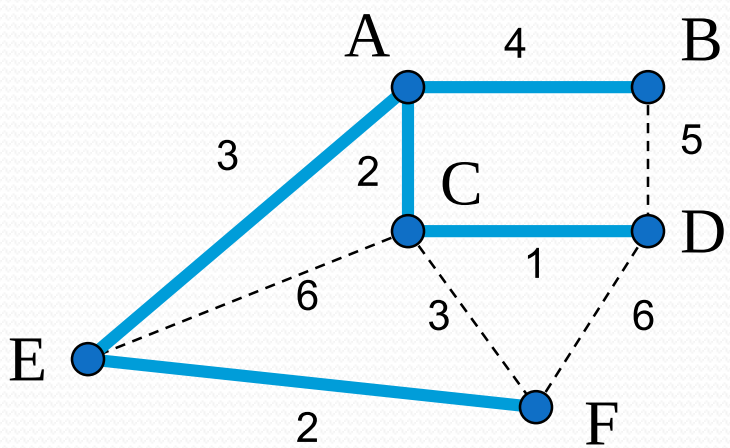
- Another issue is that a graph node may have **any number of edges leaving** it. A BST node only has 0,1, or 2 branches going out.

4. Minimal Spanning Tree

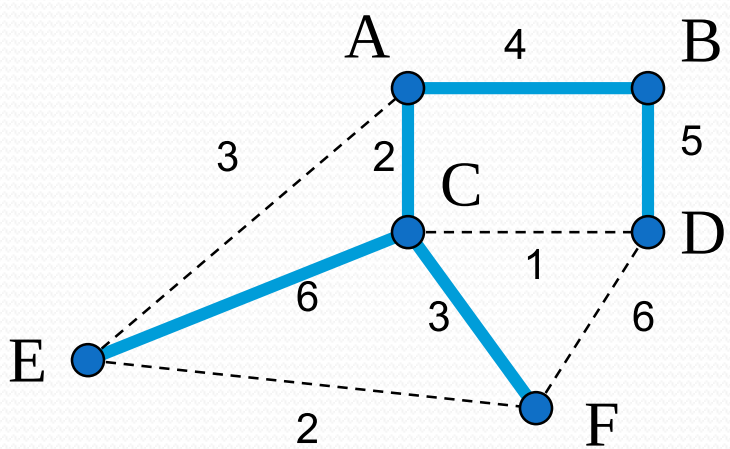
- A minimal spanning tree T is a subgraph of a *weighted* graph G which contains all the verticies of G *and* whose edges have the minimum summed weight.
- Example weighted graph G:



- A *minimal* spanning tree (weight = 12):



- A *non-minimal* spanning tree (weight = 20):



4.1. Prim's Algorithm

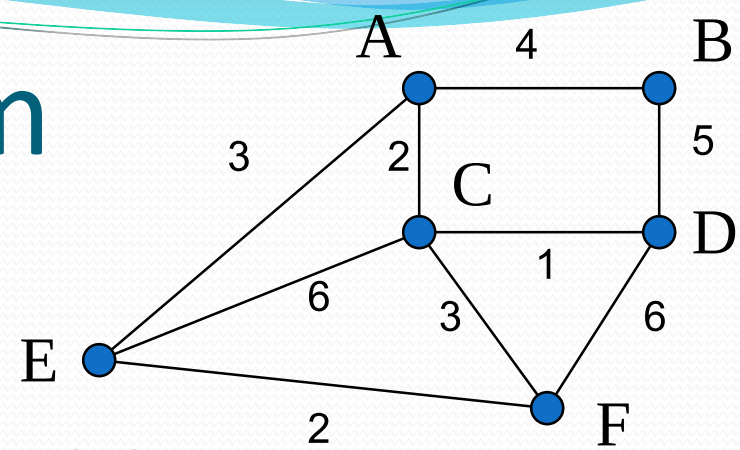
- Prim's algorithm finds a minimal spanning tree T by iteratively adding edges to T.

- At each iteration, a minimum-weight edge is added that does not create a cycle in the current T.

Robert Clay Prim
(1921 –)

Still alive!!

Informal Algorithm



- For the graph G.
- 1) Add any vertex to T
  - e.g A, T = {A}
- 2) Examine all the edges leaving {A} and add the vertex with the smallest weight.
  - edge weight
    (A,B) 4
    (A,C) 2
    (A,E) 3
  - add edge (A,C), T becomes {A,C}

continued

- 3) Examine all the edges leaving {A,C} and add the vertex with the smallest weight.
  -

edge	weight	edge	weight
(A,B)	4	(C,D)	1
(A,E)	3	(C,E)	6
(C,F)	3		

  - add edge (C,D), T becomes {A,C,D}

continued

- 4) Examine all the edges leaving {A,C,D} and add the vertex with the smallest weight.
  -

edge	weight	edge	weight
(A,B)	4	(D,B)	5
(A,E)	3	(C,E)	6
(C,F)	3	(D,F)	6

  - add edge (A,E) or (C,F), it does not matter
  - add edge (A,E), T becomes {A,C,D,E}

continued

- 5) Examine all the edges leaving {A,C,D,E} and add the vertex with the smallest weight.
  -

edge	weight	edge	weight
(A,B)	4	(D,B)	5
(C,F)	3	(D,F)	6
(E,F)	2		

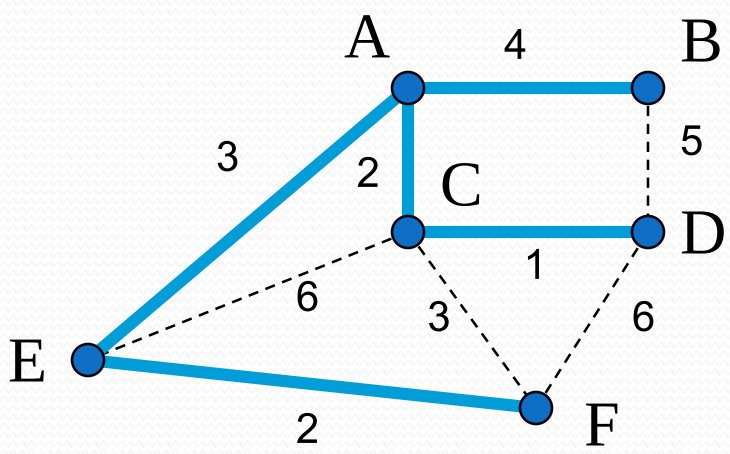
  - add edge (E,F), T becomes {A,C,D,E,F}

continued

- 6) Examine all the edges leaving {A,C,D,E,F} and add the vertex with the smallest weight.
  - edge weight edge weight
    (A,B) 4 (D,B) 5
  - add edge (A,B), T becomes {A,B,C,D,E,F}
- All the verticies of G are now in T, so we stop.

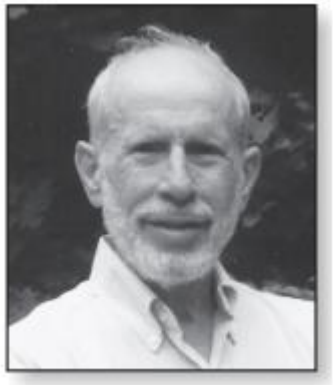
continued

- Resulting minimum spanning tree (weight = 12):



4.2. Kruskal's Minimum Spanning Tree Algorithm

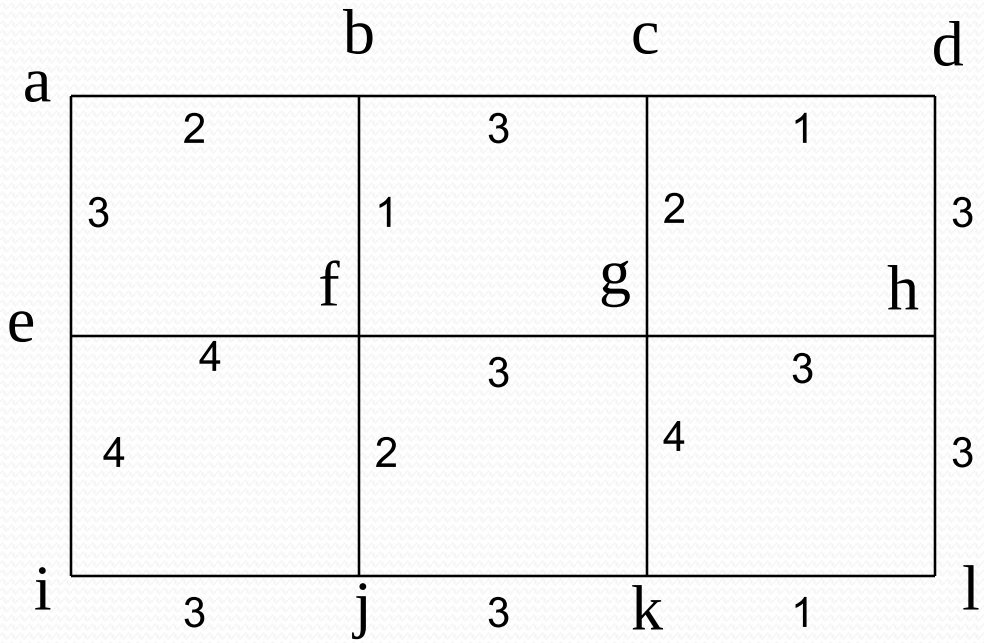
- At the start, the minimum spanning tree T consists of all the verticies of the weighted graph G, but no edges.
- At each iteration, add an edge e to T having minimum weight that does not create a cycle in T.
- When T has n-1 edges, stop.



Joseph Bernard Kruskal
(1928 – 2010)

Example

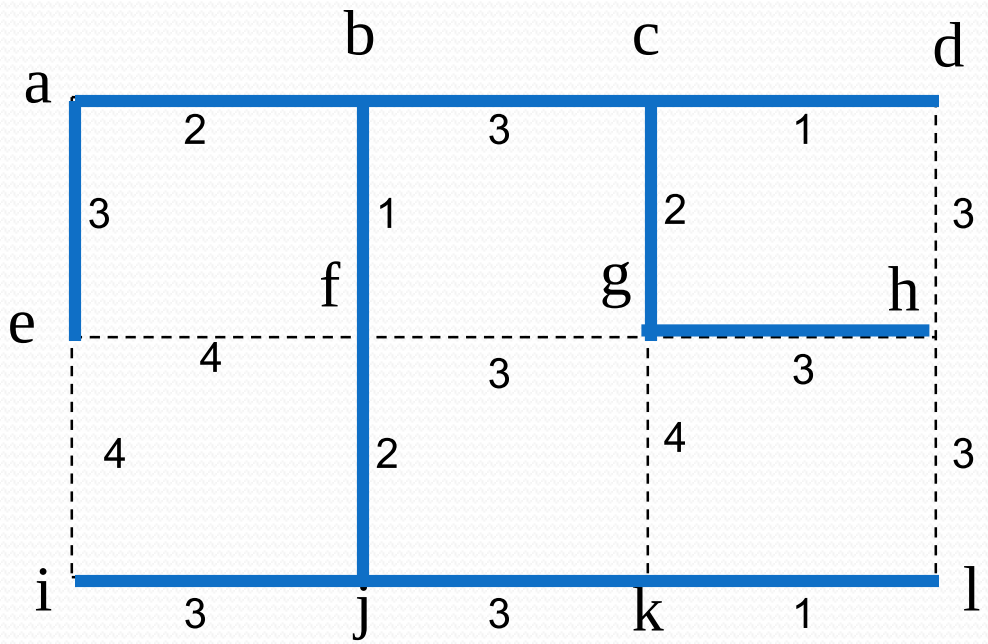
- Graph G:



iter	edge
1	(c,d)
2	(k,l)
3	(b,f)
4	(c,g)
5	(a,b)
6	(f, j)
7	(b,c)
8	(j,k)
9	(g,h)
10	(i, j)
11	(a,e)

continued

- Minimum spanning tree (weight = 24):



4.3. Difference between Prim and Kruskal

- Prim's algorithm chooses an edge that *must* already be connected to a vertex in the minimum spanning tree T.
  - there's always **one growing tree**

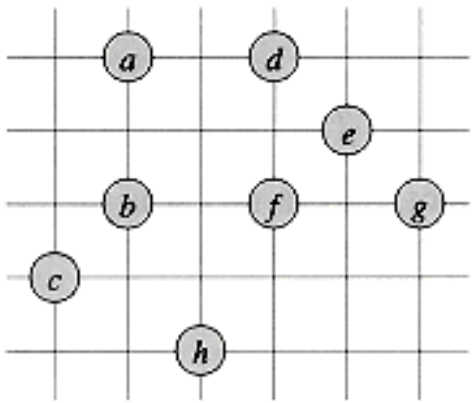
- Kruskal's algorithm can choose an edge that *may not* already be connected to a vertex in T.
  - there may be **several little trees** which come together at the end to make a single tree

5. The Travelling Salesman Problem Again

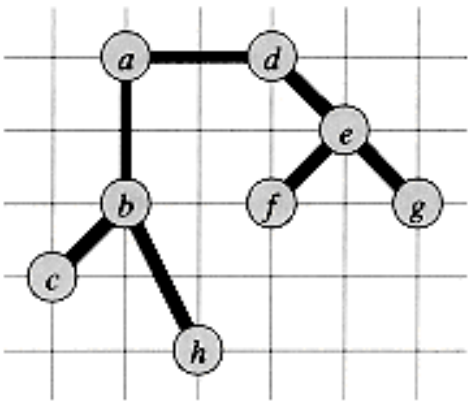
- Find a Hamiltonian cycle of a weighted undirected graph G with minimum cost.

- The following algorithm computes a fairly good tour of G, using **Prim's** algorithm, and a **preorder** walk of its minimal spanning tree.

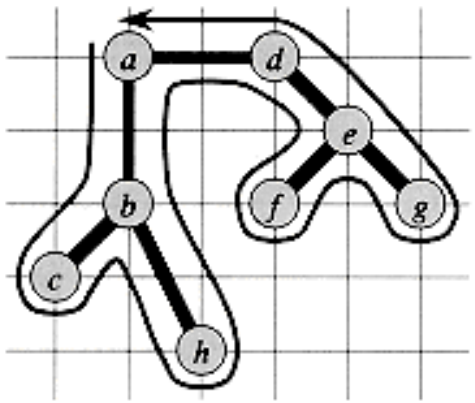
Approx TSP Tour in Pictures



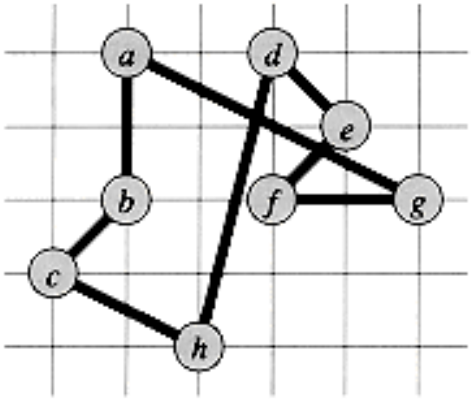
(a)



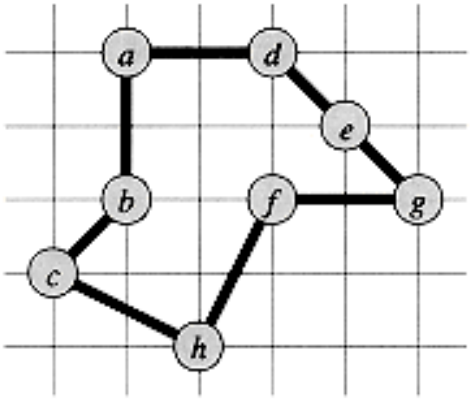
(b)



(c)



(d)



(e)

- a) The vertices in the graph
  - the weighted edges are not shown to simplify things
- b) MST calculated using Prim's algorithm
  - a is the root
- c) A preorder walk of the spanning tree
- d) The tour obtained by visiting the vertices in the order given by the preorder walk, bypassing already visited nodes.
  - cost == 20
- e) The best tour actually costs 15

6. More Information

- *Discrete Mathematics and its Applications*
  Kenneth H. Rosen
  McGraw Hill, 2007, 7th edition
  - chapter 11, sections 11.1, 11.4, 11.5